

Despliegue de una Aplicación Web Utilizando CI/CD con AWS Lambda y API Gateway

Empresa: NexaCloud

Contexto: NexaCloud es una empresa emergente de tecnología que ofrece servicios de atención al cliente a través de una plataforma web. Dado que la empresa está en crecimiento, quiere optimizar los costos de infraestructura y busca implementar una arquitectura serverless para su backend con despliegues automatizados que mejoren la eficiencia del equipo de desarrollo.

Necesidad: La empresa necesita una solución donde los usuarios puedan ingresar solicitudes de soporte a través de un formulario web. Estas solicitudes deben ser almacenadas en una base de datos, y el equipo de soporte debe poder acceder a ellas fácilmente. TechLife también quiere reducir el tiempo entre desarrollo y despliegue, automatizando el proceso de integración y entrega continua (CI/CD).

Objetivo del proyecto: Desarrollar y desplegar una aplicación web que permita a los usuarios enviar solicitudes de soporte, y utilizar un pipeline CI/CD para asegurar que las nuevas características sean desplegadas automáticamente a AWS Lambda y gestionadas a través de API Gateway. Los datos enviados por los usuarios deben almacenarse en DynamoDB, y el equipo de soporte debe tener acceso para visualizar las solicitudes.

Descripción del Proyecto

Este proyecto requiere el desarrollo de una aplicación web que permita a los usuarios enviar solicitudes de soporte. El backend de la aplicación estará basado en funciones serverless utilizando AWS Lambda, y API Gateway se encargará de gestionar las peticiones HTTP. La aplicación web debe ser desplegada utilizando un pipeline CI/CD, que automatice la integración y el despliegue de nuevas funcionalidades.

Requisitos del Proyecto

Backend:

AWS Lambda: Crear funciones Lambda que manejen las solicitudes HTTP para enviar formularios de soporte y recuperar los datos almacenados.

API Gateway: Configurar API Gateway para gestionar las solicitudes HTTP hacia las funciones Lambda. Asegurarse de manejar correctamente los métodos HTTP.

DynamoDB: Crear una tabla DynamoDB para almacenar las solicitudes de soporte enviadas por los usuarios. Las entradas deben incluir información como nombre, correo electrónico, descripción del problema, y timestamp.

Frontend:

HTML/CSS/JavaScript: Crear una página web con un formulario que permita a los usuarios ingresar información básica (nombre, correo, mensaje) y enviar una solicitud.

Integración con el Backend: La aplicación web debe llamar a la API expuesta en API Gateway para enviar y recibir información.

Pipeline CI/CD:

GitHub Actions o AWS CodePipeline: Crear un pipeline de CI/CD que automatice el despliegue de la aplicación a Lambda y cualquier cambio en el código del frontend.

Automatización de Despliegues: Cada vez que se realice un commit en el repositorio, el pipeline debe realizar pruebas (si aplica) y luego desplegar automáticamente el código en AWS.

Notificaciones: Configurar notificaciones en caso de fallas en el pipeline para que los estudiantes puedan ser alertados si algo falla.

Criterios de Éxito

- La aplicación debe ser funcional y accesible a través de un navegador web.
- Las solicitudes enviadas deben ser almacenadas correctamente en DynamoDB y ser recuperables a través de la API.
- El pipeline CI/CD debe funcionar correctamente, con despliegues automáticos al hacer un commit en la rama principal del repositorio.
- La solución debe ser escalable y optimizada en términos de costos, aprovechando la arquitectura serverless.
- Implementar autenticación básica para que solo usuarios autenticados puedan ver las solicitudes de soporte.
- Configurar AWS CloudWatch para monitorear las invocaciones de Lambda y crear alarmas en caso de fallas.

Entrega Final

Repositorio GitHub: Los estudiantes deben entregar un repositorio con todo el código (frontend y backend) y el pipeline CI/CD configurado.

Documentación: Incluir un archivo README con instrucciones detalladas de cómo ejecutar y desplegar la solución.

Presentación: Preparar una presentación explicando cómo se construyó la solución, los retos encontrados, y cómo se implementó el CI/CD.

Arquitectura: Incluir un diagrama visual de la arquitectura mostrando cómo interactúan los componentes del sistema.